CS6886 — Assignment 5

Optimising Attention on a CPU

Adhil Ahmed (da24s023) • November 8, 2026

GitHub: https://github.com/starkAhmed43/CS6886_A5

1 Q1 — Machine, peaks and ridge points

The machine is an Apple M1 with 4 P (performance) cores and 4 E (efficiency) cores. Table 1 lists the facts, also given in `machine_spec.txt`. An FMA (fused multiply-add) counts as 2 FLOP (floating-point operations). GFLOP/s means billion FLOP per second. NEON is the ARM 128-bit SIMD (single instruction, multiple data) instruction set.

Table 1: Machine facts.

Item	Value
CPU	Apple M1 (MacBookPro17,1), Firestorm (P) + Icestorm (E), ARMv8.5-A
Cores	8 physical, 8 logical (no SMT: simultaneous multithreading; one thread per core); 4 P + 4 E
Clock	Max 3.204 GHz (P), 2.064 GHz (E). No published base clock; lowest step is 0.600 GHz on both P & E cores. Under load: P, 4 threads 2.81–2.97 GHz; 8 threads, P and E mixed, 2.07–2.51 GHz
SIMD	NEON, 128 bit = 4 FP32 (32-bit floating point) lanes
FMA units per core	4 (P), 4 (E)
L1 data cache	128 KB per P core, 64 KB per E core
L2 cache	12 MB shared by the 4 P cores, 4 MB shared by the 4 E cores
L3 cache	Not available
RAM	16 GB LPDDR4X, 8 channels, 4266 MT/s (mega-transfers per second; public value, assumed)
Memory bandwidth	68.3 GB/s (computed: 4266 MT/s × 16 B)

The peak is cores × GHz × FMA units × lanes × FLOP per FMA. Lanes are the numbers one vector instruction handles at once. A NEON test with 16 independent accumulators (registers that each hold a running sum) gives 0.25 cycles per FMA, which is 4 FMA per cycle.

$$\text{P core} = 1 \times 3.204 \times 4 \times 4 \times 2 = 102.5 \text{ GFLOP/s}$$
$$\text{E core} = 1 \times 2.064 \times 4 \times 4 \times 2 = 66.0 \text{ GFLOP/s}$$
$$\text{All cores} = 4 \times 102.5 + 4 \times 66.0 = 674.3 \text{ GFLOP/s}$$

The bandwidth comes from the spec: a 128-bit (16 B) bus at 4266 MT/s.

$$\text{Memory Bandwidth} = 4266 \times 16/1000 = 68.3 \text{ GB/s}$$

The ridge point is peak divided by bandwidth:

$$\text{One P core} = 102.5/68.3 = 1.50 \text{ FLOP/B}$$
$$\text{All cores} = 674.3/68.3 = 9.87 \text{ FLOP/B}$$

2 Q2 — The naive code

The naive code runs at 2.2–2.4 GFLOP/s, about 2% of the one-core peak (Table 2). N is the number of tokens and d is the head dimension. $S = QK^\top/\sqrt{d}$ is the $N \times N$ score matrix and $P = \text{softmax}(S)$. The two products are $S = QK^\top$ (called $QK^\top$) and $O = PV$ (called PV). Time grows with N^2: doubling N at $d = 64$ multiplies the time by 4.17, then by 3.97. Time is close to linear in d: doubling d at $N = 4096$

Table 2: Naive attention on a single thread.

N	d	Time (ms)	GFLOP/s	% of peak
1024	64	111.4	2.41	2.35%
2048	64	464.7	2.31	2.25%
4096	64	1845.0	2.33	2.27%
4096	128	3947.5	2.18	2.13%

Table 3: Naive time per phase (ms) and share of the phase total.

N, d	$QK^\top$	Softmax	PV	Sum
1024, 64	21.6 (19.9%)	3.4 (3.1%)	83.5 (77.0%)	108.5
2048, 64	86.6 (19.2%)	13.6 (3.0%)	349.9 (77.7%)	450.2
4096, 64	346.8 (19.1%)	54.4 (3.0%)	1411.8 (77.9%)	1812.9
4096, 128	995.0 (25.5%)	55.3 (1.4%)	2844.1 (73.0%)	3894.3

multiplies it by 2.14. FLOPs equal $4N^2d$, so the GFLOP/s stay almost constant. Table 3 splits the time by phase.

Where the time goes: PV takes about 77%, $QK^\top$ about 19% and softmax 1.4–3.1%.

PV **is latency-bound:** PV is one chain of dependent FMAs (each FMA waits for the result of the one before it): `acc += a * b`. The compiler cannot split it, because `-O3` without `-ffast-math` keeps the order of float additions. PV runs at 4.0 cycles per FMA (at 3.15 GHz), which equals the 4-cycle FMA latency (the time one FMA takes to finish). $QK^\top$ runs at 1.0 cycle per FMA, because its chains are only 64 long and the core overlaps several dot products.

Chain test: With four independent accumulators, PV drops from 1342 ms to 340 ms (1.0 cycle per FMA). $QK^\top$ drops from 331 ms to 192 ms (0.56 cycle per FMA). The result differs by at most 4×10^{-6} (relative).

Stride test: The PV loop reads `V[j*d + k]` with stride d (the distance in memory between two reads). A transposed copy of V takes 1.2 ms. With one chain, PV stays at 1372 ms. With four accumulators, strided V takes 705 ms and transposed V takes 340 ms. We did not measure why.

Size of S**:** The rate stays flat while S grows from 4 MB to 64 MB, and the L2 cache is 12 MB. $QK^\top$ stays at 0.96–0.97 cycle per FMA and PV at 3.7–3.9 cycles per FMA.

3 Q3 — Single-thread optimisation

The **best** variant is 28.9–39.4× faster than naive and reaches 68–84% of the single-core peak (Table 4). `make check` passes for every variant on every input, with a maximum relative error from 8.3×10^{-7} to 6.2×10^{-6} (limit 10^{-4}).

loops: The code transposes K once and reorders the loops. The inner loop now runs over contiguous data, so the compiler vectorises both products (uses SIMD instructions). The speedup is 7.3–9.9×. It still stores the full S, makes three passes over it, calls scalar `expf` (the exponential function, one number per call), and loads and stores a row element for each FMA. These costs probably limit it to 17–21% of peak.

tiled: The code adds a 4×16 register tile (a small block of results kept in registers; 16 accumulators). It also blocks the columns of S and rows of V in blocks of 256 for the cache. The speedup is 15.8–22.5×. It still stores the full S and runs softmax as a separate scalar pass. Both probably limit it. We did not measure how much each costs.

fused: The code takes 4 query rows at a time and keeps a $4 \times N$ score buffer in L1. It never forms the $N \times N$ matrix. The $QK^\top$ step uses a 4×16 register block. The PV step has no register tile, so each FMA needs a load and a store. This makes fused slower than tiled (26.8 against 36.8 GFLOP/s at $N = 4096$, $d = 64$). It also still uses scalar `expf`.

best: The code works in flash style (attention computed in blocks without storing S). It takes blocks of 16 query rows and 256 keys, and keeps the 16×256 score tile (16 KB) in L1. An online softmax (softmax updated block by block with a running maximum and sum) means it never stores S. A 4×16 NEON

Table 4: Single-thread variants.

N	d	Variant	Time (ms)	GFLOP/s	% peak	Speedup
1024	64	naive	111.435	2.41	2.4%	1.00×
		loops	15.347	17.49	17.1%	7.26×
		tiled	7.278	36.88	36.0%	15.31×
		fused	9.949	26.98	26.3%	11.20×
		best	3.857	69.60	67.9%	28.89×
2048	64	naive	464.681	2.31	2.3%	1.00×
		loops	60.863	17.64	17.2%	7.63×
		tiled	27.954	38.41	37.5%	16.62×
		fused	39.767	27.00	26.3%	11.69×
		best	13.493	79.58	77.6%	34.44×
4096	64	naive	1844.958	2.33	2.3%	1.00×
		loops	243.577	17.63	17.2%	7.57×
		tiled	116.687	36.81	35.9%	15.81×
		fused	160.319	26.79	26.1%	11.51×
		best	53.880	79.71	77.8%	34.24×
4096	128	naive	3947.510	2.18	2.1%	1.00×
		loops	399.447	21.50	21.0%	9.88×
		tiled	175.759	48.87	47.7%	22.46×
		fused	263.896	32.55	31.8%	14.96×
		best	100.202	85.73	83.6%	39.40×

micro-kernel (the small inner loop that fills the register tile) with 16 accumulators runs both products: 16 FMAs per 5 loads. A degree-5 polynomial computes `exp2` on 4 lanes, and the code divides once per output row. This reaches 77.8% of peak at $N = 4096$, $d = 64$. The NEON FMA execution units and the exp cost probably limit it.

4 Q4 — OpenMP

The `omp` variant runs the 16-row blocks of best in parallel with `schedule(dynamic, 1)`. Threads share only the read-only packed K. Table 5 gives the time and the speedup over one thread. Efficiency E is the speedup divided by the number of threads. The table also gives the min–max time of 7 runs for $N = 4096$, $d = 64$.

Table 5: `omp`: median time (ms), speedup over one thread and efficiency E. T is the number of threads.

	1024, 64			2048, 64			4096, 64				4096, 128		
T	Time	Speedup	E	Time	Speedup	E	Time	Speedup	E	Min–max	Time	Speedup	E
1	3.686	1.00	1.00	14.199	1.00	1.00	57.661	1.00	1.00	56.9–58.7	110.252	1.00	1.00
2	1.942	1.90	0.95	7.572	1.88	0.94	30.170	1.91	0.96	29.4–33.3	57.683	1.91	0.96
3	1.304	2.83	0.94	5.040	2.82	0.94	19.859	2.90	0.97	19.8–24.0	37.317	2.95	0.98
4	1.213	3.04	0.76	4.709	3.02	0.75	18.614	3.10	0.77	16.8–20.0	35.232	3.13	0.78
5	1.134	3.25	0.65	4.481	3.17	0.63	17.185	3.36	0.67	15.8–20.1	33.230	3.32	0.66
6	0.921	4.00	0.67	3.519	4.03	0.67	14.104	4.09	0.68	13.9–14.3	27.780	3.97	0.66
7	1.085	3.40	0.49	3.950	3.59	0.51	15.416	3.74	0.53	12.3–21.4	28.261	3.90	0.56
8	1.013	3.64	0.45	3.798	3.74	0.47	15.769	3.66	0.46	11.6–19.9	30.360	3.63	0.45

1–3 threads: Scaling is almost linear. At $N = 4096$, $d = 64$, E is 0.96 at 2 threads and 0.97 at 3 threads.

4–6 threads: E falls to 0.77 at 4 threads and 0.68 at 6 threads, but the time still drops. Six threads give the best median time on every input: speedup 4.09 at $N = 4096$, $d = 64$.

7–8 threads: There is no further gain, and the run time varies a lot. At 8 threads the 7 runs for $N = 4096$, $d = 64$ span 11.6 to 19.9 ms.

Peak: The highest result is 309.2 GFLOP/s, with 6 threads at $N = 4096$, $d = 128$. This is 46% of the 674.3 GFLOP/s all-core peak and 75% of 4×102.5.

Why it bends: The cause is probably the E cores. An E core peaks at 66.0 GFLOP/s, against 102.5 for a P core. macOS may also move threads between core types.

Schedule: At 8 threads, `schedule(dynamic, 1)` was 12–23% faster than `static`. Fast P cores can take more blocks.

5 Q5 — Roofline

Under this model, every variant lies right of its ridge point (the point where bandwidth stops limiting speed), so DRAM bandwidth limits none of them (Figure 1, Table 6). Instead, instruction issue, the FMA execution units and the core mix limit them.

FLOP: Every variant does $4N^2d$ FLOP: two products of $2N^2d$ each.

Bytes: Naive, loops and tiled make 7 passes over S ($4N^2$ bytes), plus $16Nd$ bytes for Q, K, V and O. Fused, best and omp move only the $16Nd$ bytes. Arithmetic intensity (AI) is the number of FLOP per byte moved to or from RAM. For $N = 4096$, $d = 64$:

$$\begin{aligned}
\text{FLOP} &= 4 \times 4096^2 \times 64 = 4.295 \times 10^9 \\
\text{Bytes (naive)} &= 7 \times 4 \times 4096^2 + 16 \times 4096 \times 64 = 473{,}956{,}352 \\
\text{AI (naive)} &= 4.295 \times 10^9 / 473{,}956{,}352 = 9.06 \\
\text{AI (fused)} &= 4.295 \times 10^9 / (16 \times 4096 \times 64) = 1024
\end{aligned}$$

Roof: The roof is the smaller of the peak and arithmetic intensity × 68.3 GB/s. At an arithmetic intensity of 9.06, the bandwidth roof is 619 GFLOP/s, so the 102.5 GFLOP/s peak is the roof.

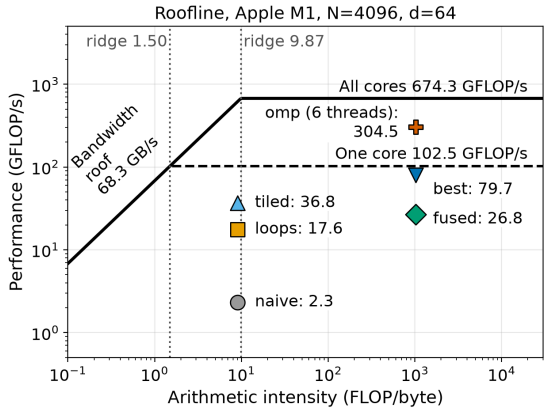


Figure 1: Roofline at $N = 4096$, $d = 64$. Bandwidth is 68.3 GB/s.

Table 6: Roofline points, $N = 4096$, $d = 64$

Variant	Threads	AI (FLOP/B)	GFLOP/s	Roof	% of roof
naive	1	9.06	2.33	102.5	2.3%
loops	1	9.06	17.63	102.5	17.2%
tiled	1	9.06	36.81	102.5	35.9%
fused	1	1024.00	26.79	102.5	26.1%
best	1	1024.00	79.71	102.5	77.8%
omp	6	1024.00	304.52	674.3	45.2%

What limits each variant:

- Naive: the scalar dependent chain (Q2).
- Loops: probably the L1 loads and stores per FMA, and scalar `expf`.
- Tiled: probably the micro-kernel code and the scalar `expf` pass.
- Fused and best: the NEON FMA execution units and the exp cost. Fused also loses to its memory-based PV update.
- OMP: the core count, the slower E cores and scheduling (Q4).